UNIVERSIDADE FEDERAL DE SERGIPE

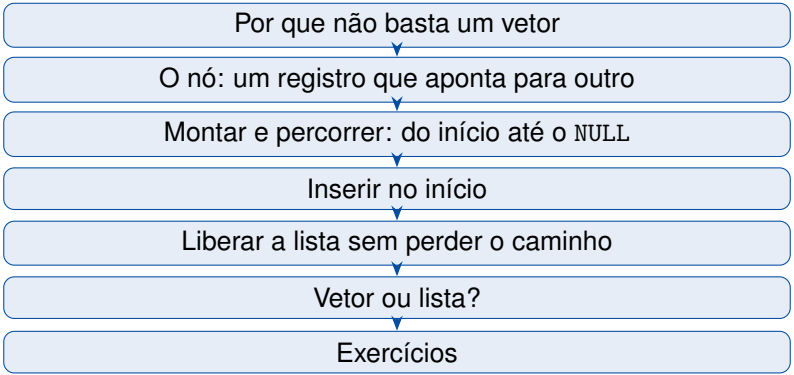
Listas encadeadas

Uma introdução: o nó, o encadeamento e o caminho até o `NULL`

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje



A pergunta de hoje

O problema: guardar uma coleção que muda o tempo todo

Até aqui

A `agenda.c` guardava os contatos num vetor no heap, que crescia com `realloc`. Funciona — enquanto os contatos entram *no fim*.

E se cada contato novo precisar entrar no começo? Ou no meio, em ordem alfabética?

O que vamos descobrir

Um vetor exige que os elementos fiquem **lado a lado** na memória. Abrir mão disso — cada elemento num bloco próprio, apontando para o próximo — é a ideia da **lista encadeada**, a primeira estrutura de dados dinâmica do curso.

A pergunta de hoje

Como guardar uma sequência sem que os elementos precisem estar vizinhos na memória — e o que isso custa?

O programa de hoje: `lista.c` (1/3)

```c
/* lista.c --- uma lista encadeada */
#include <stdio.h>
#include <stdlib.h>

typedef struct no {
    int valor;
    struct no *prox;  /* ou NULL */
} No;
```

Cada valor da lista mora num `No`, alocado com `malloc`.

```c
/* devolve o novo inicio da lista */
No *insere_inicio(No *ini, int valor)
{
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) {
        perror("malloc");
        exit(1);
    }
    novo->valor = valor;
    novo->prox = ini;
    return novo;
}
```

lista.c (2/3): percorrer e liberar

```c
void imprime(const No *ini)
{
    for (const No *p = ini; p != NULL;
         p = p->prox)
        printf("%d -> ", p->valor);
    printf("NULL\n");
}

int conta(const No *ini)
{
    if (ini == NULL)
        return 0;
    return 1 + conta(ini->prox);
}
```

```c
void libera(No *ini)
{
    while (ini != NULL) {
        No *prox = ini->prox; /* antes! */
        free(ini);
        ini = prox;
    }
}
```

lista.c (3/3): o main

```c
int main(void)
{
    No *lista = NULL;           /* vazia */
    for (int i = 1; i <= 4; i++)
        lista = insere_inicio(lista, 10 * i);
    imprime(lista);
    printf("%d nos, %zu bytes cada\n",
           conta(lista), sizeof(No));
    libera(lista);
    return 0;
}
```

A lista começa vazia: `lista` vale `NULL`.

Cada volta do laço cria um nó com o valor $10 \cdot i$ e guarda em `lista` o ponteiro que `insere_inicio` devolve.

O que este programa imprime?

Antes de compilar, escreva sua resposta

1. Em que ordem os valores 10, 20, 30 e 40 aparecem na primeira linha?
2. Quantos nós `conta` encontra?
3. Quantos bytes tem cada `No`? Um `int` tem 4 bytes e um ponteiro, 8.
4. Quantas vezes `malloc` é chamado? E `free`?

Compile, execute e compare

```
$ gcc -Wall -Wextra lista.c -o lista
$ ./lista
40 -> 30 -> 20 -> 10 -> NULL
4 nos, 16 bytes cada
```

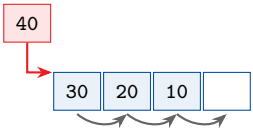
Duas surpresas

- a ordem saiu **invertida**;
- 16 bytes, e não $4 + 8 = 12$.

Quatro `malloc`, um por nó, e quatro `free`. Ao longo da aula, cada surpresa ganha uma explicação.

Por que não basta um vetor

Inserir no começo de um vetor custa quanto?



cada um anda uma casa para a direita

Pense antes de passar o slide

Para pôr 40 na posição 0, os k elementos que já estão lá precisam se deslocar.

Inserindo n valores, sempre no começo, quantos deslocamentos são feitos no total? E se n dobrar, o tempo dobra?

A resposta: medindo com `vetor_vs_lista.c`

n	deslocamentos	vetor	lista
10 000	$5{,}0 \times 10^7$	0,008 s	0,000 s
100 000	$5{,}0 \times 10^9$	0,365 s	0,001 s
200 000	$2{,}0 \times 10^{10}$	1,838 s	0,002 s
400 000	$8{,}0 \times 10^{10}$	6,109 s	0,003 s

compilado com `-O2`; deslocamento com `memmove`

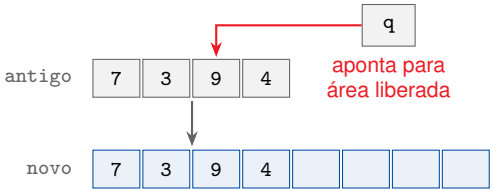
$0 + 1 + \cdots + (n-1) = \frac{n(n-1)}{2}$ deslocamentos.

Dobrou n, o vetor levou **4 vezes** mais tempo: o custo cresce com n^2.

A lista não desloca ninguém: cada inserção cria um nó e acerta dois ponteiros.

Fonte: Cormen et al. (2009), §10.2 [3]; medido com `vetor_vs_lista.c`

E o `realloc`? O bloco pode mudar de endereço



Quando não há espaço logo depois do bloco, o `realloc` copia tudo para outro lugar e libera o antigo. Inserindo 200 mil valores, o bloco mudou de lugar **12 vezes**.

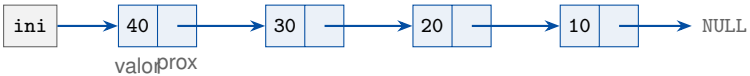
Todo ponteiro guardado para dentro do vetor antigo (como `q = &v[2]`) passa a apontar para memória liberada.

Fonte: ISO/IEC 9899:2018, §7.22.3.5 [5]; medido com `vetor_vs_lista.c`

Definição: lista encadeada

Lista encadeada

Uma sequência de **nós**, cada um alocado separadamente. Cada nó guarda um valor e o **endereço do próximo**; o último guarda `NULL`. Basta conhecer o endereço do primeiro nó — o **início** — para chegar a todos.



Intuição

Uma caça ao tesouro: cada pista diz onde está a próxima. A ordem está nas *setas*, não na posição de cada nó na memória.

Fonte: Knuth (1997), §2.2.3 [8]; Celes et al. (2004), cap. 10 [2]

De onde veio a ideia

As listas encadeadas surgiram em 1955–56, com Allen Newell, Cliff Shaw e Herbert Simon, na linguagem IPL, criada para o *Logic Theory Machine*: um programa que provava teoremas e precisava de estruturas que crescessem e mudassem de forma durante a execução.

Antes disso, a memória de um programa era organizada em posições fixas — como um vetor.

O que ficou

A mesma ideia está por trás de pilhas, filas, árvores e grafos — os próximos temas do curso.

Fonte: Newell & Shaw (1957) [9]; Newell, Shaw & Simon (1957) [10]; Knuth (1997), §2.2.3, notas históricas [8]

Recapitulando: por que não basta um vetor

O que aprendemos

- num vetor, os elementos ficam lado a lado: inserir no começo desloca todos;
- inserir n valores no começo custa $\approx n^2/2$ deslocamentos;
- o `realloc` pode mover o bloco e invalidar ponteiros antigos;
- numa lista encadeada, cada nó mora num bloco próprio e a ordem é dada pelos ponteiros.

O nó

Um registro que aponta para outro do mesmo tipo

```
typedef struct no {
    int valor;
    struct no *prox;
} No;
```

Pense antes de passar o slide

1. Por que o campo é `struct no *prox`, e não `struct no prox` (sem o `*`)?
2. Por que não escrever `No *prox` lá dentro?
3. Por que o registro tem nome (`no`) se já existe o apelido `No`?

Definição: estrutura auto-referenciada

Estrutura auto-referenciada

Um registro com um campo que é **ponteiro** para o próprio tipo do registro.

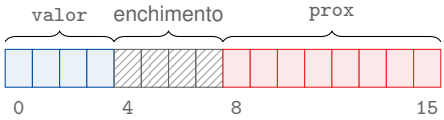
Sem o `*`: um `no` conteria um `no`, que conteria outro `no`... O tamanho seria infinito, e o compilador recusa.

Com o `*`: o campo guarda só um endereço — 8 bytes, qualquer que seja o tamanho do nó.

O nome `struct no`: dentro das chaves, o apelido `No` ainda não existe — o `typedef` só termina no `;`. Já `struct no` existe desde a primeira linha, como um tipo *incompleto*: não dá para criar um, mas dá para apontar para um.

Fonte: Kernighan & Ritchie (1989), §6.5 e §6.7 [6]; ISO/IEC 9899:2018, §6.7.2.3 [5]

Por que 16 bytes, e não 12?



É o alinhamento da aula de registros: um ponteiro de 8 bytes precisa começar num endereço múltiplo de 8. O compilador põe 4 bytes de enchimento depois de `valor`.

Numa lista de `int`, cada valor de 4 bytes custa **16**: a lista troca memória por flexibilidade.

Fonte: Bryant & O'Hallaron (2015), §3.9.3 [1]; ISO/IEC 9899:2018, §6.7.2.1 [5]

Recapitulando: o nó

O que aprendemos

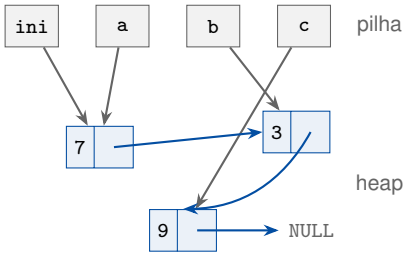
- o nó é um registro com o valor e um ponteiro para o próximo nó;
- o campo precisa ser ponteiro: um nó não pode conter um nó;
- dentro das chaves, use `struct no`: o apelido ainda não existe;
- cada nó carrega o custo do ponteiro (e do enchimento).

Montar e percorrer

Três nós montados à mão

```
No *a = malloc(sizeof *a);
No *b = malloc(sizeof *b);
No *c = malloc(sizeof *c);
a->valor = 7;  a->prox = b;
b->valor = 3;  b->prox = c;
c->valor = 9;  c->prox = NULL;
No *ini = a;
```

`p->campo` é o mesmo que `(*p).campo`:
“siga o ponteiro e pegue o campo”.



Os nós ficam onde o `malloc` quiser; a ordem 7, 3, 9 está nas setas.

Fonte: King (2008), §17.5 [7]

Percorrer: o que acontece com `p`?

```
for (const No *p = ini; p != NULL;
     p = p->prox)
    printf("%d -> ", p->valor);
printf("NULL\n");
```

Pense antes de passar o slide

Com a lista $7 \rightarrow 3 \rightarrow 9$:

1. quantas vezes a condição `p != NULL` é testada?
2. que valores `p` assume?
3. o que acontece se a lista for vazia (`ini == NULL`)?
4. e se o último nó tiver lixo no lugar de `NULL`?

A resposta: `p` anda de seta em seta

teste	`p` aponta para	imprime
1	nó 7	`7 ->`
2	nó 3	`3 ->`
3	nó 9	`9 ->`
4	`NULL`	sai do laço

n nós, $n + 1$ testes. Com a lista vazia, o primeiro teste já falha e só sai `NULL`: o caso vazio funciona *sem* `if` especial.

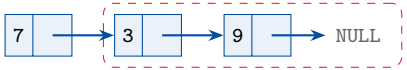
Sem o `NULL` no fim, `p` segue um endereço de lixo: o programa imprime bobagem ou cai. O `NULL` é a única forma de saber que a lista acabou.

Fonte: Feofiloff (2009), cap. 4 [4]; Kernighan & Ritchie (1989), §6.5 [6]

Uma lista é vazia, ou um nó seguido de uma lista

```c
int conta(const No *ini)
{
    if (ini == NULL)       /* vazia */
        return 0;
    return 1 + conta(ini->prox);
}
```

A definição da lista já é recursiva — e `conta` só a segue: caso base na lista vazia, chamada no resto.



Lembre da aula passada

Um nó = um registro na pilha. Com 300 mil nós, sem `-O2`, `conta` estoura a pilha.

Fonte: Feofiloff (2009), cap. 4 [4]; medido com `lista.c`

Recapitulando: montar e percorrer

O que aprendemos

- `p->campo` segue o ponteiro e acessa o campo;
- percorrer é andar `p = p->prox` até `p == NULL`;
- o mesmo laço trata a lista vazia, sem caso especial;
- a lista é uma definição recursiva — mas a recursão gasta um registro por nó.

Inserir no início

Por que a ordem saiu invertida?

`main` chamou, nesta ordem:

```
insere_inicio(lista, 10)
insere_inicio(lista, 20)
insere_inicio(lista, 30)
insere_inicio(lista, 40)
```

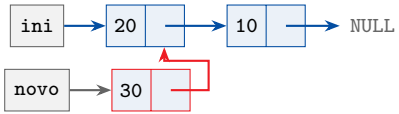
e `imprime` mostrou `40 -> 30 -> 20 -> 10`.

Pense antes de passar o slide

1. Desenhe a lista depois de cada chamada.
2. Por que `insere_inicio` *devolve* um ponteiro?
3. O que acontece se o `main` escrever só `insere_inicio(lista, 50);`, sem atribuir?

Inserir no início: dois ponteiros, nesta ordem

1. `novo->prox = ini;`



2. `return novo;` e `lista = ...` no `main`



Na ordem contrária — primeiro mudar o início, depois ligar — o endereço do antigo primeiro nó se perderia, e com ele a lista inteira.

São sempre os mesmos passos, com 3 ou 3 milhões de nós: custo **constante**.

Fonte: Celes et al. (2004), cap. 10 [2]; Cormen et al. (2009), §10.2 [3]

A resposta: `ini` é uma cópia

```
No *lista = NULL;
insere_inicio(lista, 50);  /* ERRADO */
imprime(lista);            /* NULL */
```

A função recebe uma **cópia** de `lista`. Mudar `ini` lá dentro não muda `lista` aqui fora — a mesma passagem por valor da aula de ponteiros.

Duas consequências

- `lista` continua `NULL`;
- o nó criado fica no heap sem ninguém apontando para ele: um **vazamento**.

Por isso a função devolve o novo início, e quem chama *atribui*: `lista = insere_inicio(lista, 50);`

Fonte: Kernighan & Ritchie (1989), §5.2 [6]; King (2008), §17.5 [7]

Recapitulando: inserir no início

O que aprendemos

- o novo nó aponta para o antigo início; depois ele vira o início;
- a ordem dos dois passos importa: ligar antes de mudar o início;
- quem chama precisa guardar o ponteiro devolvido;
- o custo não depende do tamanho da lista — e por isso a ordem sai invertida.

Liberar a lista

Um `libera` mais curto

```c
/* libera_errada.c */
void libera(No *ini)
{
    for (No *p = ini; p != NULL;
         p = p->prox)
        free(p);
}
```

Três linhas, o mesmo laço do `imprime`, com `free` no lugar do `printf`.

Pense antes de passar o slide

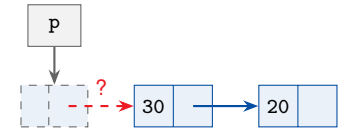
1. Esta versão libera os quatro nós?
2. Em que momento é lido `p->prox`: antes ou depois do `free(p)`?
3. O programa vai funcionar, cair, ou depende?

Execute e compare

```
$ gcc -Wall -Wextra libera_errada.c -o libera_errada      # nenhum aviso
$ ./libera_errada
Abort trap: 6
$ gcc -g -fsanitize=address libera_errada.c -o libera_errada-asan
$ ./libera_errada-asan
==4117==ERROR: AddressSanitizer: heap-use-after-free on address 0x602000000158
READ of size 8 at 0x602000000158 thread T0
    #0 0x000102040950 in libera libera_errada.c:22
0x602000000158 is located 8 bytes inside of 16-byte region
freed by thread T0 here:
    #1 0x000102040920 in libera libera_errada.c:23
```

No macOS, o alocador derrubou o programa. Em outro sistema, o mesmo código pode cair de outra forma — ou parecer funcionar.

A resposta: o `p->prox` vem depois do `free`



Depois de `free(p)`, o bloco volta para o alocador, que pode reescrevê-lo. “8 bytes dentro de uma região de 16” é exatamente o campo `prox`.

A correção: guarde o caminho antes

```
No *prox = ini->prox;
free(ini);
ini = prox;
```

Usar um bloco depois do `free` é comportamento indefinido: o padrão não promete nada. O *sanitizer* transforma “depende” em uma mensagem com a linha do erro.

Fonte: ISO/IEC 9899:2018, §7.22.3.3 [5]; Serebryany et al. (2012) [11]

Recapitulando: liberar a lista

O que aprendemos

- cada `malloc` de nó precisa do seu `free`: um laço que visita todos;
- depois do `free`, o nó não pode mais ser lido — nem o `prox`;
- guarde o próximo numa variável *antes* de liberar o atual;
- o compilador não avisa; use `-fsanitize=address` para encontrar o erro.

Vetor ou lista?

Cada estrutura é boa em uma coisa

operação	vetor	lista encadeada
acessar o *k*-ésimo elemento	imediato (`v[k]`)	percorre *k* nós
inserir no começo	desloca *n*	**constante**
inserir no fim	constante*	percorre *n* nós
memória por `int`	4 bytes	16 bytes
elementos vizinhos na memória	sim	não

* em média, com `realloc` dobrando a capacidade

Elementos vizinhos aproveitam melhor a *cache* do processador: somar 10 milhões de `int` levou 0,005 s no vetor e 0,025 s na lista — 5 vezes mais, mesmo com os nós alocados em sequência.

Fonte: Cormen et al. (2009), §10.2 [3]; Bryant & O'Hallaron (2015), §6.2 [1]; medido com `percorre.c`

Recapitulando: vetor ou lista?

O que aprendemos

- vetor: acesso direto por índice, mas inserir fora do fim desloca elementos;
- lista: inserir no começo é barato, mas chegar ao k-ésimo exige percorrer;
- a lista gasta mais memória por elemento e usa pior a cache;
- a escolha depende das operações que o programa mais faz.

Mãos à obra

Altere o programa

Tarefa 1

Em `imprime`, mostre também o endereço de cada nó (`%p`, com `(void *) p`). Os nós estão lado a lado? A distância entre eles é de 16 bytes?

Tarefa 2

Troque o `4` do laço do `main` por `1000000` e tire o `imprime`. Preveja: `conta` funciona sem `-O2`? E com? Depois reescreva `conta` com um laço.

Tarefa 3

Escreva `int soma(const No *ini)` duas vezes: com laço e recursiva. Teste com a lista vazia.

Exercícios (1/2)

Exercício 1 — o maior valor

`int maximo(const No *ini)` devolve o maior valor da lista. O que fazer quando a lista é vazia? Escolha e documente no comentário.

Exercício 2 — do vetor para a lista

`No *de_vetor(const int *v, int n)` cria uma lista com os valores de `v` **na mesma ordem**, usando só `insere_inicio`. Dica: por qual ponta do vetor começar?

Exercício 3 — imprimir de trás para frente

`void imprime_invertida(const No *ini)`, recursiva, sem criar outra lista. Onde fica o `printf`: na ida ou na volta?

Fonte: Feofiloff (2009), cap. 4 [4]; Celes et al. (2004), cap. 10 [2]

Exercícios (2/2)

Exercício 4 — listas iguais

`int iguais(const No *a, const No *b)` devolve 1 se as duas listas têm os mesmos valores, na mesma ordem, e 0 caso contrário. Cuidado com listas de tamanhos diferentes.

Exercício 5 — uma cópia independente

`No *copia(const No *ini)` devolve uma lista nova, na mesma ordem, com nós próprios. Libere a original e mostre que a cópia continua funcionando.

Exercício 6 — `k`-ésimo

`const No *kesimo(const No *ini, int k)` devolve o nó da posição `k` (a partir de 0), ou `NULL` se ela não existir. Quantos nós são visitados para `k = n - 1`?

Fonte: King (2008), §17.5 [7]; Tenenbaum et al. (1995), cap. 4 [12]

Desafio

Contatos sem limite, com nomes do tamanho certo

Leia nomes da entrada padrão, um por linha, até o fim do arquivo, e guarde-os numa lista encadeada de

```
typedef struct contato { char *nome; struct contato *prox; } Contato;
```

- cada nome é lido num vetor local de 100 posições e **copiado** para um bloco do `malloc` com o tamanho exato;
- imprima os nomes em ordem inversa à da leitura e, depois, na ordem da leitura;
- no fim, libere tudo: dois `free` por nó — em que ordem?

Com `-fsanitize=address`, nenhum erro e nenhum vazamento.

Fonte: King (2008), §17.2 e §17.5 [7]

Fechamento

O que mudou hoje

A sequência deixou de depender da posição na memória: a ordem agora está nos ponteiros. Inserir no começo ficou barato — com 3 ou 3 milhões de nós.

O preço

Mais memória por elemento, nada de acesso por índice, e um `malloc` e um `free` para cada nó.

Para levar

- o nó é um registro com valor e `struct no *prox`;
- percorrer: `p = p->prox` até `NULL`;
- inserir no início: ligar o novo nó, depois devolvê-lo como início;
- liberar: guardar o próximo antes do `free`.

Próxima aula: busca, remoção e inserção em qualquer posição.

Referências I



Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seções 6.2 (localidade), 9.9 (alocação dinâmica) e 9.11 (erros comuns de memória).



Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
Introdução a estruturas de dados: com técnicas de programação em C.
Elsevier, Rio de Janeiro, 2004.
Capítulo 10: listas encadeadas.

Referências II


Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, Cambridge, MA, 3 edition, 2009.
Seção 10.2: linked lists.


Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 4: listas encadeadas.

Referências III


ISO/IEC.
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seções 6.7.2.3 (tags e tipos incompletos) e 7.22.3 (gerenciamento de memória).


Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.
Seções 6.5 (estruturas auto-referenciadas) e 6.7 (`typedef`).

Referências IV


K. N. King.
C Programming: A Modern Approach.
W. W. Norton, New York, 2 edition, 2008.
Seções 17.4 (deallocating storage) e 17.5 (linked lists).


Donald E. Knuth.
The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, Reading, MA, 3 edition, 1997.
Seção 2.2.3: alocação encadeada (com notas históricas).

Referências V



Allen Newell and J. C. Shaw.
Programming the logic theory machine.
In *Proceedings of the Western Joint Computer Conference*, pages 230–240, 1957.
Origem das listas encadeadas (linguagem IPL).



Allen Newell, J. C. Shaw, and Herbert A. Simon.
Empirical explorations of the logic theory machine: A case study in heuristic.
In *Proceedings of the Western Joint Computer Conference*, pages 218–230, 1957.
O *Logic Theory Machine*, para o qual a IPL foi criada.

Referências VI



Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Detecta uso de memória após `free`.



Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
Capítulo 4: filas e listas (listas ligadas).